Weekly Project Log

Week 1 — 2026-06-13 to 2026-06-19

13 June. I fixed the general direction of my EPQ and set up the folders for the report, research notes, data, code, appendix, presentation and Production Log. My first idea was simply about using machine learning to predict cryptocurrency, but I quickly realised that this was too broad. It was not clear which cryptocurrency I would use, what I would predict or how I would decide whether a model was good.

17 June. I narrowed the project to Bitcoin volatility forecasting. I chose volatility instead of trying to predict the exact Bitcoin price because volatility has a clearer connection to risk and can be compared using numerical errors. I started reading about log returns, realised volatility, rolling historical volatility and GARCH. For the machine-learning side, I chose Random Forest and LSTM because they use different ideas: Random Forest can learn nonlinear relationships from prepared features, while LSTM is designed for sequences.

I originally planned to use Yahoo Finance, but I changed the data source to Hyperliquid. This gave me daily OHLCV data from one specific Bitcoin perpetual-futures market. I downloaded the first dataset, calculated daily log returns and used a rolling window to create the volatility value that the models would predict. I also produced the first results table so that I could see whether the whole process worked from data collection to model comparison.

At this stage, I decided that I should not judge the models only by the lowest error. I also wanted to compare how easy they were to explain, how long they took to run and whether the extra complexity actually gave a useful improvement. This changed the project from a simple model ranking into a comparison of accuracy and practicality.

What I learned. The biggest lesson from the first week was that the question had to be much more specific. Once I had fixed the asset, the target and the main models, the project became easier to plan. I also found that choosing the target was just as important as choosing the model.

Next step. Prepare the data more carefully, build the baseline methods first and then compare them with the machine-learning models using the same dates and the same error measures.

Week 4 — 2026-07-04 to 2026-07-10

9–10 July. I refreshed the Hyperliquid daily data and reran the models because I did not want to keep writing the report using results from an older dataset. I checked that the dates were in order and that the price fields were sensible before using the data. I then recalculated the log returns and 30-day realised-volatility target.

I worked on the main comparison between rolling historical volatility, GARCH, lagged linear regression and Random Forest. I used a time-based split rather than randomly mixing the rows. This mattered because a forecasting model should only learn from information that would have been available before the date being predicted. I compared the models using MAE, MSE and RMSE, then updated the results section and the comparison chart.

While writing the results, I noticed that simply listing the error values did not explain enough. I added comments on what each model was doing. Rolling volatility was the easiest baseline to understand. GARCH was more specialised because it modelled volatility clustering. Linear regression showed whether the lagged features already contained useful information in a simple form, while Random Forest tested whether nonlinear relationships improved the forecast.

What I learned. Refreshing the data can change the figures, so the report and the model outputs have to be checked together. I also became more careful about using exactly the same test period for every model. Otherwise, a lower error would not be a fair comparison.

Next step. Complete the LSTM model, check the date alignment of every prediction and add more tests to see whether the model ranking was stable.

Week 5 — 2026-07-11 to 2026-07-17

12 July. I refreshed the dataset again. Later, I found that the newest daily candle was still open when it was downloaded. Its closing price, volume and trade count were therefore incomplete. I changed the data collection step so that a daily candle would only be kept after its end time had passed.

13 July. I completed the LSTM comparison and reorganised the code into separate parts for data preparation, features, models, evaluation and outputs. This made it easier to rerun the same process without manually changing several files.

The most important problem I found was a date-alignment error in the GARCH output. The predictions had been connected using reset row numbers instead of their actual dates, so a forecast could be compared with the wrong target day. I changed the output to match predictions by date, reran all the models and rewrote the parts of the report that depended on the earlier ranking.

14 July. I continued checking the calculations. I corrected the order used in the GARCH update so that the current day's movement did not affect its own forecast. I also checked the rolling-volatility calculation and made sure that the scaling used for the LSTM was fitted only on the earlier training data, not on the later validation period.

After these corrections, I added several checks instead of relying on one result. I compared 14-day and 30-day volatility targets, split the test period into earlier and later halves, tested low-, medium- and high-volatility periods and used four expanding time windows. I also ran the LSTM with three random seeds and checked the Random Forest feature importance. These tests helped me see whether the main conclusion depended on one particular setting.

What I learned. This week showed me that a result can look realistic even when the dates or calculations are wrong. The checks were more important than making the model look successful. I also learned that correcting an earlier result is part of the research process and should be explained rather than hidden.

Next step. Run the complete pipeline one more time with the latest finished daily data and use the checked results for the final discussion and conclusion.

Week 6 — Beginning 2026-07-18

20 July. I carried out the final data refresh. Hyperliquid returned 1,241 daily rows. One row was the day that was still in progress, so I removed it and kept 1,240 completed daily candles ending on `2026-07-

19`. I reran the data-quality checks for missing fields, date order, daily spacing, positive prices, OHLC consistency, volume and trade count.

After preparing the features and volatility target, the final modelling table contained 1,195 rows. I kept 950 earlier rows for training and used 245 later rows for testing. The test period started on `2025-11-16`, so the models were compared on the same later section of the data without random shuffling.

The final 30-day RMSE ranking was:

1. GARCH(1,1): `0.00098502`
2. Lagged linear regression: `0.00140087`
3. Rolling historical volatility: `0.00142744`
4. LSTM: `0.00174351`
5. Random Forest: `0.00232370`

GARCH also ranked first with the 14-day target and in the repeated expanding-window tests. The LSTM and Random Forest did not beat the simpler rolling baseline overall. This was not the result I expected at the beginning, but it answered the project question clearly: for this dataset and this setup, the extra complexity of the two machine-learning models was not justified by better accuracy.

I ran the complete automated test set and all 39 tests passed. I then used the checked results to finish the discussion and conclusion, making sure that I did not claim the result would apply to every Bitcoin market or every possible machine-learning model.

What I learned. My final result depended more on careful data handling and fair testing than on choosing the most advanced model. I also became much more cautious about dates, incomplete market data and information from the future entering an earlier stage of the model.

Next step. Finish the presentation, explain the main corrections and results clearly, and prepare for questions about why the machine-learning models did not perform better.